Inheritance Basics

The is-a relationship, extends, super, overriding, and protected

In Classes, Objects, and Interfaces, we built classes that use other classes (a `Mechanic` uses a `Car`) and classes that contain other classes (a `Garage` has a list of `Car` objects). This unit adds the third relationship, *inheritance*, where one class is a more specialized version of another. These first notes cover the vocabulary, how to write a subclass, what it receives from its superclass, how to replace an inherited method, and how access levels work once more than one class is involved. Parts 2 through 4 build on the same example, a small payroll program. For reading, see Core Java, Volume I, Chapter 5, Section 5.1.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. Three Relationships Between Classes

You have already met two of the three ways classes relate to each other. Here are all three together.

Table 1: The three relationship types among classes.

Relationship	Meaning	Example
Dependence (uses-a)	A class uses or manipulates another class, usually through a parameter or a local variable	A `Mechanic` uses a `Car` when it inspects one
Aggregation (has-a)	A class keeps an object of another class in one of its fields	A `Garage` has a list of `Car` objects
Inheritance (is-a)	A class is a more specialized version of a more general class	A `Manager` is an `Employee`

A quick way to check whether inheritance fits is to say the sentence out loud. "A manager is an employee" sounds right, so `Manager` can inherit from `Employee`. "A garage is a car" sounds wrong, so `Garage` should hold cars, not inherit from `Car`. The is-a sentence is necessary but not always enough. The hierarchy should be natural to what you are representing, and whether it is useful depends on the requirements of the program. Part 3 shows a case where a hierarchy that looked fine had to be reorganized once a new requirement arrived.

2. Vocabulary

Table 2: Inheritance terms. Each row lists names that mean the same thing.

Term	Also called	Meaning
Superclass	Base class, parent class	The more general class that is inherited from
Subclass	Derived class, child class	The more specialized class that inherits
`extends`		The keyword that makes one class a subclass of another
Class hierarchy	Inheritance hierarchy	The tree formed by all the superclass and subclass links

The names "superclass" and "subclass" come from set theory, not from size. The set of all managers is a subset of the set of all employees. A subclass object usually holds *more* data than a superclass object, not less, because it keeps everything it inherited and adds its own.

Figure 1 shows the hierarchy we build across this unit. Each arrow points from a subclass to its superclass, which is the direction the `extends` keyword points in the code.

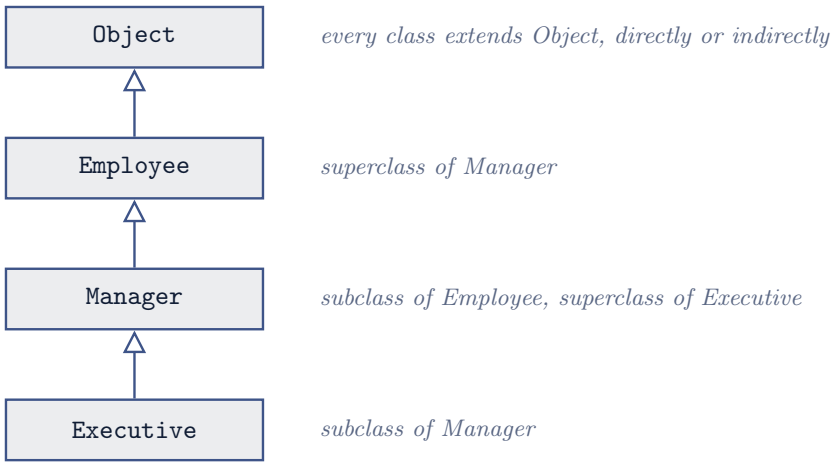


Figure 1: The payroll hierarchy as it stands at the end of Part 2. The hollow triangle is the UML symbol for inheritance and points at the superclass. In Part 3 the hierarchy grows: `Employee` becomes abstract, and `SalariedEmployee` and `HourlyEmployee` join it.

Shapes a hierarchy can take

You will see these names in other books and online. Java classes support the first three. The last two are possible only with interfaces, and Part 4 explains why.

Table 3: Kinds of inheritance and whether Java classes support them.

Kind	Meaning	Java classes?
Single	A class inherits from one superclass	Yes
Multilevel	A class inherits from a class that itself inherits from another (`Executive`, `Manager`, `Employee`)	Yes
Hierarchical	Several classes inherit from the same superclass	Yes
Multiple	A class inherits from more than one superclass	No; only through interfaces
Hybrid	A mix of two or more of the kinds above	Only when the "multiple" part uses interfaces

Java classes use *single inheritance*: each class has exactly one direct superclass. If you do not write `extends`, the superclass is `Object`. A long chain is still single inheritance, because every class in it has one parent. An `Executive` ends up as a combination of everything above it: part `Object`, part `Employee`, part `Manager`, and part `Executive`.

3. A First Subclass

Here is the superclass. It reuses an idea from the review notes: a static counter hands out IDs.

```
public class Employee {
    private static int nextId = 1;

    private final String id;
    private String name;
    private double annualSalary;

    public Employee(String name, double annualSalary) {
        if (annualSalary < 0) {
            throw new IllegalArgumentException("negative salary: " + annualSalary);
        }
        this.id = "E" + nextId;
        nextId++;
        this.name = name;
        this.annualSalary = annualSalary;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getAnnualSalary() { return annualSalary; }

    public void raiseSalary(double percent) {
        annualSalary = annualSalary + annualSalary * percent / 100;
    }

    public double getMonthlyPay() {
```

```
        return annualSalary / 12;
    }

    @Override
    public String toString() {
        return id + " " + name + ", monthly pay " + String.format("%.2f", getMonthlyPay
    ());
    }
}
```

A manager is an employee who also earns a yearly bonus. Instead of copying every line of `Employee`, we write only what is different:

```
public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double annualSalary, double bonus) {
        super(name, annualSalary);   // build the Employee part first
        this.bonus = bonus;
    }

    public double getBonus() { return bonus; }
    public void setBonus(double bonus) { this.bonus = bonus; }

    @Override
    public double getMonthlyPay() {
        return super.getMonthlyPay() + bonus / 12;
    }

    @Override
    public String toString() {
        return super.toString() + ", bonus " + String.format("%.2f", bonus);
    }
}
```

A subclass can do three things with what it inherits:

- **Use it as is.** `Manager` never mentions `getId`, `getName`, or `raiseSalary`, yet every `Manager` has them.
- **Add to it.** The `bonus` field and the `getBonus` and `setBonus` methods exist only in `Manager`. An `Employee` object does not have them.
- **Replace it.** `getMonthlyPay` and `toString` are *overridden*: the subclass supplies its own version, which runs instead of the inherited one.

```
Employee ada = new Employee("Ada", 60000);
Manager grace = new Manager("Grace", 90000, 12000);
System.out.println(ada);
System.out.println(grace);
System.out.println(grace.getId() + " is " + grace.getName());   // inherited methods
```

```
E1 Ada, monthly pay 5000.00
E2 Grace, monthly pay 8500.00, bonus 12000.00
E2 is Grace
```

Look closely at Grace's monthly pay. `Employee.toString` calls `getMonthlyPay()`, and for Grace it printed 8500.00, which includes the bonus. Even though that line of code is written inside `Employee`, it ran `Manager`'s version, because the object is a `Manager`. That is polymorphism, and it is the subject of Part 2.

4. What a Subclass Inherits

In the earlier unit, we described every object by three characteristics: state (its fields), behavior (its methods), and identity (what makes it a distinct object). Inheritance passes along state, behavior, and *type*, but not identity.

Table 4: What a subclass receives from its superclass.

Aspect	Inherited	Not inherited
State	Every instance field becomes part of the subclass object, including `private` ones. Code in the subclass can name the `public` and `protected` fields, and the package-private ones when both classes are in the same package.	Direct access to `private` fields. The data is there, but the subclass must reach it through the superclass's methods.
Behavior	`public` and `protected` methods, and package-private methods within the same package	`private` methods and constructors
Type	The superclass's type, and the types of every interface the superclass implements. A `Manager` can go anywhere an `Employee` is expected.	
Identity		Each object is its own object. Creating a `Manager` does not create a separate `Employee` object.

Private fields are inherited as data, not as access

This is the most common point of confusion, partly because two true statements sound like they disagree. The Java Language Specification says private members are not inherited. Yet every `Manager` object clearly holds a name and a salary. Both are true. The formal rule is about *names*: code in `Manager` cannot refer to `annualSalary`. The object still contains the field, because `Employee`'s methods need it.

```
// Inside Manager:
public double getMonthlyPay() {
    return annualSalary / 12 + bonus / 12;   // does not compile
}
```

```
error: annualSalary has private access in Employee
```

The fix is to ask the superclass for its part of the answer, which is what the real `getMonthlyPay` does with `super.getMonthlyPay()`. That keeps `Employee` in charge of its own data, which is encapsulation working the way it should.

this and super

Table 5: The two references that help inside a subclass.

Use	`this`	`super`
Field	`this.bonus`: a field of this object	`super.name`: a field declared in the superclass (only if it is accessible)
Method	`this.getBonus()`: the version for this object's actual class	`super.getMonthlyPay()`: the superclass's version, even if this class overrides it
Constructor	`this(...)`: another constructor in the same class	`super(...)`: a constructor in the superclass

`super` is not a reference to a separate object. There is only one object. `super` tells the compiler to start looking for the method in the superclass instead of in the current class.

Constructors are not inherited

`Manager` does not get a two-argument constructor from `Employee`. Every class writes its own constructors, and the first thing each one does is call a superclass constructor with `super(...)`. If you leave that call out, Java inserts `super()` with no arguments, which fails here because `Employee` has no constructor without parameters. Part 3 covers constructors in detail.

5. Overriding a Method

A subclass *overrides* a method when it declares a method with the same name and the same parameter types as an inherited method. Calls on a subclass object then run the subclass version.

Always write @Override

`@Override` is an *annotation*: a note to the compiler saying "this method is meant to replace an inherited one." It is optional, but write it every time. Without it, a small mistake quietly creates a brand-new method instead of an override:

```
@Override
public double getMonthlypay() {   // lowercase p: a different name
    return 0;
}
```

```
error: method does not override or implement a method from a supertype
```

Without the annotation, this would compile, and `Manager` would silently keep the inherited `getMonthlyPay`. You already used `@Override` on `toString`, `equals`, and `hashCode` in the earlier unit. Section 7 explains what those methods were overriding.

The rules

Table 6: Rules the compiler checks for an override.

Rule	Detail
Same signature	Same name and same parameter types. The parameter names do not matter.
Compatible return type	The same type, or a subclass of it. A method that returns `Employee` can be overridden by one that returns `Manager`.
Access not narrower	A `public` method must stay `public`. Making it `protected` gives "attempting to assign weaker access privileges."
Not `final`	A `final` method cannot be overridden (Part 3).
Instance methods only	`private` methods are not inherited, so they cannot be overridden. A `static` method with the same signature *hides* the inherited one instead of overriding it, which means it is not polymorphic (Part 2).

There is one more rule about exceptions that we will see in Week 11: an overriding method cannot throw new or broader checked exceptions than the method it overrides.

Overriding is not overloading

The two words sound alike and are easy to mix up, so here is a careful comparison.

Table 7: Overloading compared with overriding.

Question	Overloading	Overriding
What is the same?	The method name	The name and the parameter types
What differs?	The parameter types or count	The body
Where can it happen?	In one class, or across a superclass and a subclass	Only between a superclass and a subclass
Who chooses which runs?	The compiler, using the declared types of the arguments	The JVM, using the actual class of the object
Example	`println(int)` and `println(String)`	`Manager.getMonthlyPay()` replacing `Employee.getMonthlyPay()`

Overloading is not limited to one class. If `Manager` adds `raiseSalary(double percent, double`

`bonusPercent)`, that method overloads the inherited one-parameter `raiseSalary`, and a `Manager` has both. Part 2 shows why the "who chooses" row matters.

6. Access Levels and Packages

The review notes promised we would return to access levels once there was a subclass to talk about. Java has four levels. Three have keywords; the fourth is what you get when you write no modifier at all.

Table 8: The four access levels, from most open to most restricted.

Modifier	Same class	Same package	Subclass in another package	Any other class
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes	No
none (package-private)	Yes	Yes	No	No
`private`	Yes	No	No	No

Two things surprise people. First, `protected` is *more* open than package-private, not less: it means "my package, plus my subclasses anywhere." Second, leaving off the modifier is a real choice with real consequences, not a neutral default.

A *package* is a named group of related classes, declared with a `package` statement at the top of each file. In Eclipse, each package is a folder under `src`. Here is a small example with two packages:

```
package company;

public class Person {
    private String taxId = "123-45-6789";    // Person only
    String office = "Room 101";              // package-private: anything in company
    protected String name = "Ada";           // company, plus subclasses anywhere
    public String getName() { return name; } // everyone
}
```

```
package hr;

import company.Person;

public class Staff extends Person {          // different package, a subclass
    void show() {
        System.out.println(name);            // compiles: protected, inherited
        System.out.println(office);          // does not compile
    }
}

public class Report {                        // different package, not a subclass
    void show(Person p) {
        System.out.println(p.getName());     // compiles: public
        System.out.println(p.name);          // does not compile
```

```
    }
}
```

```
error: office is not public in Person; cannot be accessed from outside package
error: name has protected access in Person
```

(Each class goes in its own file. They are shown together here to save space.)

Which level to choose

- **Fields:** keep them `private`, as in the earlier unit. A `protected` field can be changed by every subclass anyone ever writes, and by every class in the package, so the superclass can no longer guarantee its own rules, such as "salary is never negative."
- **Methods meant for subclasses only:** `protected` is a good fit. Part 3 uses a `protected` constructor in an abstract class, and the rental example uses a `protected` method that subclasses fill in.
- **Everything else:** `public` for the methods other classes are meant to call, `private` for helpers.

Course expectation. Write an access modifier on every field and method on purpose. If you leave one off, be ready to explain why package-private access is what you wanted.

7. Object, the Root of Every Class

A class with no `extends` clause extends `java.lang.Object`. So every class you have ever written was already a subclass, and every object already had the methods in this table. That is why `toString`, `equals`, and `hashCode` needed `@Override` in the earlier unit: you were overriding `Object`'s versions.

Table 9: Commonly used methods every class inherits from Object.

Method	What Object's version does
`toString()`	Returns the class name, an `@` sign, and the hash code in hexadecimal
`equals(Object)`	Returns `true` only when both references point to the same object, the same test as `==`
`hashCode()`	Returns a number derived from the object's identity, consistent with that `equals`
`getClass()`	Returns a `Class` object describing the object's actual class; it is `final`, so it cannot be overridden

```
Class<?> c = grace.getClass();
System.out.println(c.getName());
System.out.println(c.getSuperclass().getName());
System.out.println(c.getSuperclass().getSuperclass().getName());
```

```
payroll.Manager
payroll.Employee
```

```
java.lang.Object
```

Records and enums fit into this picture too. Every record automatically extends `java.lang.Record`, and every enum automatically extends `java.lang.Enum`. Since a class can have only one superclass, and that slot is already taken, a record or an enum cannot extend anything else. That is the reason behind the rule you saw in the review notes. Both can still implement interfaces.

8. Summary

Table 10: Summary of the ideas in these notes.

Idea	Key point
Inheritance (is-a)	A subclass is a more specialized version of its superclass
`extends`	Names the one direct superclass; without it, the superclass is `Object`
Inherited	State (as data), accessible behavior, and type; not constructors, not identity
`super`	Calls the superclass's constructor or the superclass's version of a method
Overriding	Same signature in a subclass; always mark it with `@Override`
Overloading	Same name, different parameters; chosen by the compiler
`protected`	Accessible in the package and in subclasses anywhere; prefer it for methods, not fields
`Object`	The root of every hierarchy; source of `toString`, `equals`, `hashCode`, `getClass`

9. Try It

1. Remove the line `super(name, annualSalary);` from the `Manager` constructor and compile. Read the full error message, including the "required" and "found" lines, and explain what Java tried to do on your behalf.
2. Add an overload `raiseSalary(double percent, double bonusPercent)` to `Manager` that raises the salary (by calling the inherited method) and the bonus. Call both versions on a manager and confirm both exist.
3. In `Manager`, change the access of `getMonthlyPay` from `public` to `protected`. Read the error message and explain why Java refuses.